

Aerolytics: Comprehensive Viva & Technical Architecture Document

1. Project Abstract & Core Philosophy

Aerolytics is an advanced, full-stack Urban Air Quality Intelligence Platform designed for 23 major Indian cities. Standard Air Quality Index (AQI) dashboards often present static numbers without context. The core philosophy behind Aerolytics is to provide a complete "intelligence loop": **Monitor** (what is happening?), **Forecast** (what will happen?), **Attribute** (why is it happening?), and **Advise** (what should the user do about it?).

The system leverages state-of-the-art Deep Learning for forecasting, rigorous statistical methods for source attribution, a highly responsive React frontend, and an integrated Multilingual AI Chatbot.

2. External Data Sources & APIs Chosen

2.1 OpenAQ API (v3) - The Pollution Data Source

- **Where it's used:** Backend ingestion pipeline (app/services/pollution.py) and the Chatbot proxy (/api/sensor/).
- **Why it was chosen:** OpenAQ provides standard, aggregated data from official government monitoring stations (like CPCB, DPCC, TNPCB) across India. It gives granular, sensor-level data for PM2.5, PM10, NO2, CO, SO2, and O3, which is required for sub-index calculations. It is much more reliable and research-focused than commercial weather APIs.

2.2 Open-Meteo API - The Meteorological Data Source

- **Where it's used:** Backend ingestion pipeline (app/services/weather.py).
- **Why it was chosen:** Open-Meteo offers high-resolution hourly weather data (Temperature, Humidity, Wind Speed, Precipitation) entirely for free without requiring API keys. Furthermore, it provides highly accurate historical data and multi-day forecasts. Weather forecasts act as "known future inputs" which are crucial for our Machine Learning models to predict future pollution dispersion.

2.3 Google Gemini API (Gemini 2.5 Flash) - The NLP Engine

- **Where it's used:** Backend LLM service (app/services/llm.py).
 - **Why it was chosen:** Gemini 2.5 Flash offers exceptional speed and context handling. We use it to power the Chatbot. By defining a strict System Prompt, we force the LLM to output native HTML tags (like ,
) instead of Markdown, ensuring seamless integration into our custom Chatbot UI without needing a heavy Markdown parser on the frontend.
-

3. The Backend Architecture (Python / FastAPI)

3.1 Framework Selection: FastAPI

- **Why FastAPI?** Traditional frameworks like Django or Flask are synchronous by default. FastAPI

uses ASGI (Asynchronous Server Gateway Interface), making it blazingly fast. It easily handles concurrent requests for map data, charts, and chatbot LLM streams without blocking.

3.2 Data Management: Local-First CSV Buffers

- **The Problem:** Machine Learning models require a continuous 24-hour sliding window of historical data to make a prediction. Querying the OpenAQ API every time a user requests a forecast would result in massive latency and rate-limiting.
- **The Solution:** The backend uses APScheduler to run a **Staggered Ingestion Service**. Every hour, background threads fetch new data and persist it locally into "City-Specific CSV Buffers".
- **Data Cleaning:** Missing indices are forward-filled using Pandas, negative sensor glitches are clamped to 0, and timezones are aligned to IST.

3.3 State & Consistency: Rankings Cache

- **The Architecture:** The backend features a rankings.json cache. This acts as the Single Source of Truth. The /aqi/current endpoint (Dashboard) and /aqi/rankings endpoint (Map Explore Page) both read from this exact same cache.
 - **Why?** This guarantees that a city's AQI shown on the national map exactly matches its value on the detailed dashboard. The cache is refreshed automatically every 5 minutes.
-

4. Machine Learning & Statistical Engine

4.1 Forecasting: Temporal Fusion Transformer (TFT)

- **Where it's used:** Pre-computed in the background and served via /aqi/forecast. Built with PyTorch and PyTorch Forecasting.
- **Why TFT over LSTMs or standard Transformers?**
 1. **Multi-variate Capability:** It naturally handles *Static Metadata* (e.g., city location), *Time-Varying Knowns* (future weather forecasts), and *Time-Varying Unknowns* (historical pollution). Standard LSTMs struggle to separate known future variables from unknown ones.
 2. **Quantile Regression:** Instead of a single point prediction, TFT outputs a range (\$P_{10}\$, \$P_{50}\$, \$P_{90}\$). This allows the frontend to visualize a **Confidence Band** (Forecast Uncertainty).

4.2 Source Attribution: Granger Causality (Statsmodels)

- **Where it's used:** The Causal Engine (app/services/causal.py) served via /analyze/causal.
 - **The Math:** Correlation does not imply causation. We use **Vector Autoregression (VAR)** to mathematically test if the history of one variable (e.g., Wind Speed) significantly improves the forecast of our target (PM2.5).
 - **Why it was chosen:** By calculating statistical P-values across up to 6 lags, we can definitively prove directional drivers. We scale these logarithmic P-values into human descriptors like "Extremely Strong Link", providing interpretable science to non-experts.
-

5. The Frontend Architecture (React 19)

5.1 Tech Stack Justification

- **React 19 & Vite:** Chosen for modern, component-based UI development with incredibly fast hot-module-reloading during development.
- **TanStack React Query v5:** Used for state management.
 - **Why?** It automatically *dedupes requests*. If the Map Component and the Sidebar Component both request the Delhi AQI simultaneously, React Query merges them into a single network call. It also handles automatic retries on network failures.
- **Tailwind CSS 4.0 & Framer Motion:** Chosen to create a "Vibrant Design System". Generic colors are avoided; instead, dynamic HSL color scales are algorithmically generated based on the AQI severity (Good to Severe). Framer Motion handles the micro-animations that make the interface feel premium.

5.2 Key Dashboard Components

- **PollutantRiskRadar (Recharts):** Uses a spider-chart to show the relative health-risk weights of PM2.5, PM10, NO2, CO, and O3 simultaneously.
- **Forecast Visual Bridge:**
 - **The Problem:** ML model predictions at $T=0$ can slightly differ from real-time sensors, creating an ugly "cliff" on the chart.
 - **The Solution:** We implemented a linear interpolation algorithm in `/aqi/forecast` that blends the anchor (real-time AQI) with the model's prediction over a 4-hour window (80%, 60%, 40%, 20%).

5.3 Regulatory Compliance

The frontend strictly adheres to the **Indian National Air Quality Index (NAQI)** standards from the CPCB, standardizing categories from Good to Severe and dynamically updating UI colors to match the exact government specified bounds.

6. Multilingual Chatbot Architecture (chatbot.html)

The Chatbot is a localized, context-aware widget embedded into the application, designed to bridge the gap between complex data and the layman.

6.1 Real-Time Proxy & Security

- **The Problem:** Calling the OpenAQ API directly from the frontend JavaScript exposes our API keys to the public.
- **The Solution:** The chatbot calls a backend proxy (`/api/sensor/{location_id}`). This hides the API key, bypasses browser CORS restrictions, and ensures the backend can aggressively timeout if the third-party API stalls.

6.2 Client-Side Heuristics & NAQI Calculation

- To reduce server load, the chatbot contains hardcoded NAQI breakpoints (BP) and calculates the sub-indices natively in JavaScript using a custom `subIndex` interpolator function.
- It features an `AREA_MAP` object mapping dozens of localized neighborhoods (e.g., "IIT Delhi", "Punjabi Bagh", "Velachery") to their nearest physical monitoring station (e.g., "RK Puram", "Alandur").

6.3 Multilingual & Text-to-Speech (TTS)

- **Language Selection:** Users can switch between English, Hindi, Tamil, and Kannada. The Gemini LLM is prompted via SYSTEM_PROMPT to respond strictly in the selected language script.
- **TTS Engine:** We integrated the native browser window.speechSynthesis API. When the user changes language, the system searches the OS for the exact regional voice profile (e.g., hi-IN for Hindi, ta-IN for Tamil) so the pronunciation is authentic.

6.4 Chat History Context

- The Chatbot maintains an internal array (CHAT_HISTORY) of role-based objects (User vs Model). When a new message is sent to the backend /api/chat, the entire history is sent along with it. This allows the Gemini LLM to implicitly understand context (e.g., if the user asks "How bad is it in Delhi?" and then follows up with "What mask should I wear?", the AI knows the user is still asking about Delhi's specific pollution type).